

Resumen Capa de Red

☰ Comentario	
☑ Completo	☐

Capa de Red

Habilita el hecho de que se pueda enviar la información a través de la red. Cada equipo intermedio se denomina router.

- Una sola modalidad: **IP**.
- Características principales:
 - **Conmutación de paquetes** => Routing, Switching.
 - **Best-effort**. Modelo muy económico.

Idea de cliente servidor es un poco ambigua, ya que la conexión es completamente bidireccional. Ambos costados pueden iniciar la comunicación. No nos interesa mucho quién es cliente y quién servidor, nos interesa la comunicación entre extremos.

Generalidades

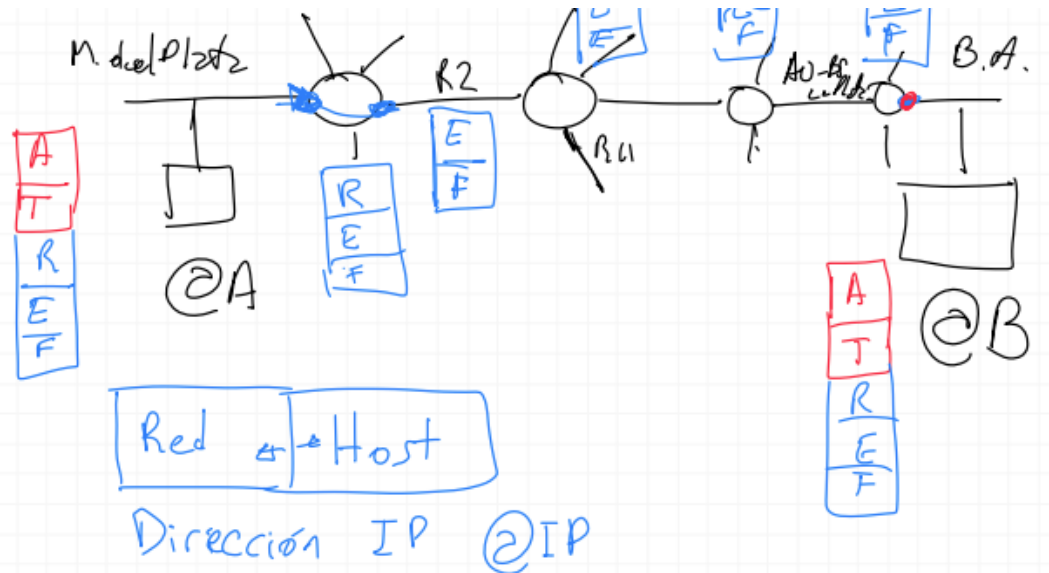
El principal rol de la capa de red es mover paquetes de un host enviador a un host receptor. Se identifican dos funciones importantes:

Forwarding: cuando un paquete llega al enlace de entrada de un router, el router debe mover el paquete al enlace de salida apropiado. Toma muy poco tiempo, típicamente en la medida de nanosegundos. Se suele implementar en hardware.

Routing: se debe determinar la ruta o el camino tomados por los paquetes a medida que viajan del enviador al receptor. Toma un poco más de tiempo, en la medida de los segundos. Se suele implementar en software.

Routing es planear todo el camino desde el sender al receiver. Forwarding es dentro de un router determinar cuál va a ser la salida.

[Es la version del libro que al profe no le gusta buscar la version que le gusta a el].



El host A tiene una cierta dirección, va a enviar paquetes por la red que le llegan a distintos routers hasta que llega a su destino final B. La cuestión importante es que hay diferentes routers en el medio, los routers son diferentes divisiones del camino.

Como sabe el paquete a donde tiene que viajar? Hay una cantidad de datos que nos llevan al destino, datos jerárquicos: el numero de red y el numero de host → con estos números nos podemos mover en internet ⇒ **dirección IP**. Lo que pasa es que operábamos en una cierta aplicación (capa de aplicación y transporte) hasta ahora, estas capas tienen sentido unicamente en los extremos. Durante el traslado del paquete en la red solo existen las capas de red, de enlace y la física. Dentro del enlace lo unico que opera es la capa enlace y la física. En los routers existe la salida por defecto, en donde si ninguna de las otras salidas corresponde a la deseada, al tomar la salida por defecto van a aparecer como opciones las anteriormente ofrecidas y otras. Los routers routean al paquete segun el campo de red de la dirección IP, decidiendo por que enlace debe salir. Un elemento clave es la tabla de forwarding, en el header de cada paquete se evalúan algunos campos con los que se indexa esta tabla. El valor almacenado en la tabla para dicho índice, indica cuál es la salida a tomar.

Conexión

- El conector elige **rutas**.
- Cuando sale, se denomina “forwarding” o “envío”.
- Routeo/Switching ==> Forwarding.
- En el **router**, hay **Red**-Enlace-Física.
- En el **cable**, sólo Enlace-Física.

Dirección IPv4

Concepto de división **red** : **host** . La division entre estos dos cambia depende de en qué parte de la red estamos(contexto de la red).

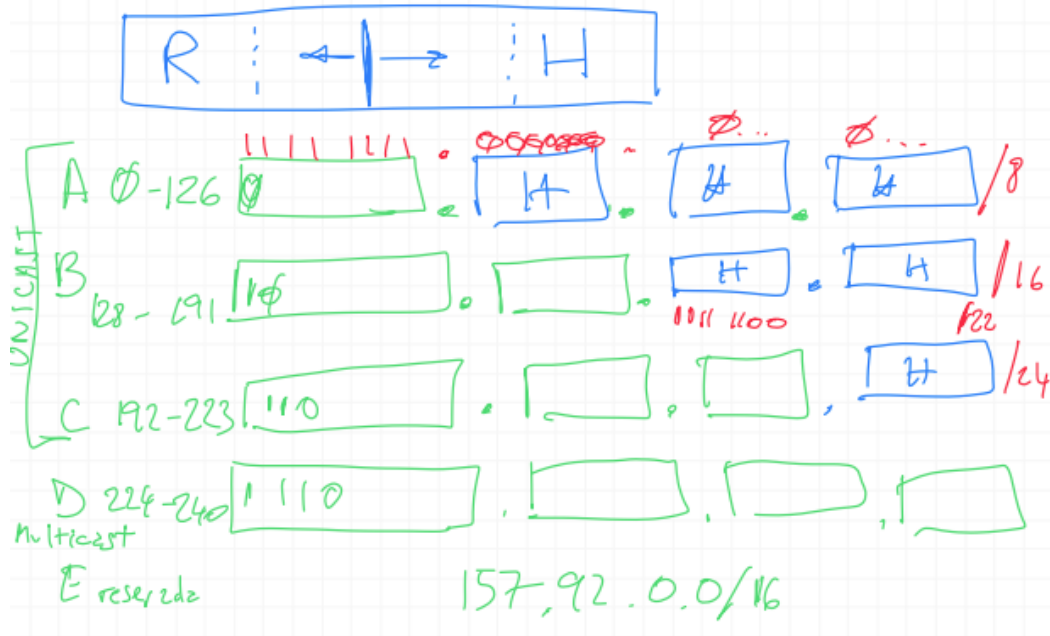
SIEMPRE tienen 4 bytes.

En el routeo con clases existia una clase A que arranca en 0 y termina en 126. El byte tiene 8 posiciones si en la primer posición pongo 0 se que el numero en binario va a estar entre 0-126. Siguiendo esta idea:

Inicios

Ruteo con las clases **A** , **B** , **C** , y **D** .

- Unicast:
 - **A** : **0-126** . Su primer byte es: 0... Ultimos 3 bytes para hosts.[Podría llegar hasta 127, pero esta es una dirección reservada].
 - **B** : **128-191** . Su primer byte es: 10... Ultimos 2 bytes para hosts.
 - **C** : **192-223** . Su primer byte es: 110.... Ultimo byte para hosts.
- Multicast
 - **D** : **224-239** . Su primer byte es: 1110...
- De 240 en adelante, están *reservadas*.
- **127.** => **localhost** .



Mascaras

Si tenemos una dirección perteneciente a la clase A y ponemos en los 3 bytes de host todos los bits en 0 y hacemos un AND lógico entre esa dirección y una dirección IP $\Rightarrow$ el resultado es el número red. Si queremos saber el número de host ponemos todos los bits del primer byte en 0 y hacemos un AND lógico $\Rightarrow$ obtenemos el número de host.

Esto descrito se denomina máscara y empieza con unos y sigue con ceros.

Máscaras: $a.b.c.d/x$, donde x es el número de bits a izquierda que van en 1 .

- La máscara es **relativa al router** en el que está en la red. Depende de en qué punto de la red está.
- Variable a lo largo de la red.

Cada router puede tener **prefijos** de distintas longitudes:

- Los agrupa por longitud.
- Intenta de los más largos a los más cortos, dentro de los que tiene.
- Convención: match del **prefijo más largo**.
- Hay caminos alternativos.

Redes privadas reservadas

No hay ningún host público que tenga estas direcciones.

- Clase A: 127.0.0.0
- Clase A: 10.0.0.0
- Clase B: 172.16.0.0 - 172.32.0.0
- Clase C: 192.168.0.0

Redes en general

- Las que tienen 1 s significan **broadcast**.
- Las que tienen 0 s significan **"this"**.

Routers

Puede suceder que un paquete quede loopeando entre routers, ya que significa que las tablas de ruteo estan mal ⇒ se puede solucionar, lo soluciona IP.

Inicialmente el router era monolítico:

- Construir las tablas.
- Hacer el forwarding.
- **Control Plane.** Armado de las tablas de forwarding.
 - Buscar rutas. Necesita conocer las rutas que existen, tener una vision global.
Particularidad: no se busca la mejor, sino que se busca una.
- **Data Plane.** Forwarding, mirar las tablas de ruteo y direccionar los paquetes.
 1. Viene una dirección IP.
 2. Calculo el prefijo.
 3. Miro la tabla. Convención de prefijo más largo.
 4. Salgo por donde tengo que salir.

No escalaría que en la tabla de forwarding haya una fila por cada dirección, ya que las direcciones son de 8 bits.. lo que se puede hacer entonces, es agrupar las direcciones

por prefijo, por ejemplo:

[img]

Si una dirección matchea con más de un prefijo, el router usa la regla del prefijo más largo (longest prefix matching rule), o sea busca el prefijo más largo que matchee con la dirección.

Además del lookup, en el router se ejecutan otras acciones importantes: procesos de la capa física y de enlace, chequeo de los campos de número de versión, checksum y time-to-live (los últimos dos se reescriben en esta etapa) y la actualización de los contadores relativos al manejo de la red

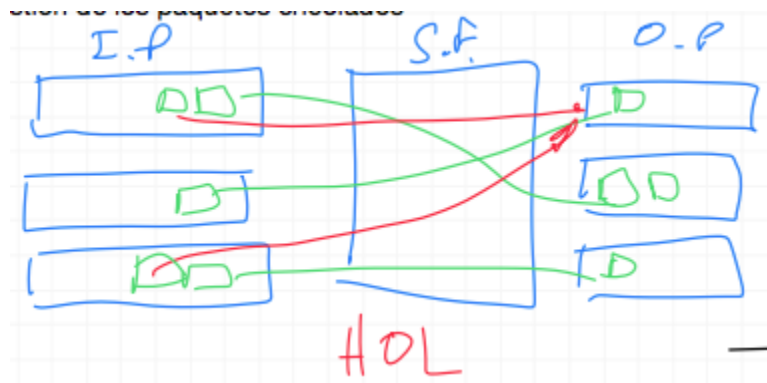
Data Plane

Arquitectura del plano de manejo de datos

- **Input port(s):** interface de entrada
- **Switch fabric(s)(conmutacion).** Cola con velocidad de salida constante.
 - Vía memoria.
 - Vía bus.
 - Vía red de interconexión.
- **Output port(s):** interface de salida

Dentro de un router se identifican los siguientes componentes:

- Puertos de entrada (input ports): Cumple la función física de "terminar" el enlace físico entrante en el router. También cumple funciones de la capa de enlace, funciones de lookup. Acá es donde se consulta a la tabla de forwarding.
- Switching fabric: Conecta los inputs con los outputs.
- Puertos de salida (output ports): Almacena los paquetes que recibe del switch fabric y los transmite al enlace de salida implementando funciones de la capa de enlace y de la capa física.



Los paquetes llegan a una tasa que depende del tamaño del paquete y de la velocidad física del enlace. Los paquetes se acumulan en la entrada si el switch no la atiende, a medida que la atiende vamos a sacar paquetes y enviarlos a la salida correspondiente. El switch fabric para que no se encolen los paquetes, debería operar a una velocidad de al menos N veces la tasa de transferencia de los enlaces de entrada, siendo N la cantidad de enlaces de entrada. Porque si te llega un paquete en simultaneo en cada uno de los enlaces tenemos que poder procesarlos todos antes de que te lleguen otros paquetes en cualquiera de los enlaces.

Supongamos que trabajamos a exactamente la misma velocidad que la máxima tasa de paquetes, si todos los paquetes que llegan van a diferentes salidas, funciona bien. Si dos paquetes van a la misma salida, depende de como funcione el switch: si lo que hace es leer una entrada y mandarla a la salida (y pasar a las siguientes entradas) en forma secuencial, esta situación nunca va a pasar. Porque el switch fabric trabaja a una velocidad tal que puede leer todas las entradas en un ciclo y que todavía no haya llegado otro paquete.

Pero si el switch fabric trabajara a una velocidad menor a esa, en la medida que tengo muchos paquetes que entran se me llenan los buffers y comienza a haber congestión. El switch fabric puede ser paralelo, por lo que podría al mismo tiempo generar 3 conexiones aunque estas deben ser **independientes**, no puede generar varias conexiones que se conecten a una misma salida. Cuando se genera esta situación en la que dos inputs van al mismo output, se llama Head Of the Line (HOL) y significa que hay dos conexiones que quieren ir al mismo lugar y se produce una colisión dentro de la estructura de switch fabric paralela. A la escala que se utiliza esto, se necesitan utilizar de forma paralela ya que es la forma de lograr velocidad.

Switch Fabric, switching

El switching se puede hacer de distintas maneras:

- Por memoria (switching via memory): los primeros routers eran computadoras tradicionales, con el switch bajo control directo de la CPU. El puerto de entrada, avisaba mediante una señal de interrupción la llegada de un paquete, luego el paquete se copiaba a la memoria del procesador. El procesador, entonces, extraía de los headers la dirección destino y a partir de la tabla de forwarding, copiaba el paquete al puerto de salida. Además, los paquetes no pueden ser transferidos al mismo tiempo, ya que la memoria solo puede hacer una operación de lectura/escritura por vez.

[img]

- Via bus: el puerto de entrada transfiere el paquete directamente al puerto de salida mediante un bus compartido, sin intervención del procesador de routing. Esto típicamente se hace pre-apendeando un label (o header) al paquete indicando el puerto de salida y transmitiendo el paquete al bus. Todos los puertos de salida reciben el paquete, pero solo se lo queda aquel que matchee con el label. Por el bus puede cruzar un paquete por vez, por ello la velocidad del router está limitada a la velocidad del bus.

[img]

- Via red interconectada (interconnection network): una solución para la limitación del bandwidth que ofrece utilizar un único bus compartido, es un switch de tipo crossbar. Consiste en $2N$ buses que conectan N puertos de entrada con N puertos de salida. Cada bus vertical intersecta cada bus horizontal en un crosspoint que puede ser abierto o cerrado en cualquier momento por el controlador del switch fabric. Cuando llega un paquete al puerto A que tiene que ser transmitido al puerto Y, el controlador cierra la intersección A-Y y envía el paquete por ese bus. Notar que aca, el puerto A podría mandar al mismo tiempo un paquete al puerto X porque son distintos buses. O sea, que se pueden enviar paquetes en paralelo. Además, es non-blocking, a menos que dos paquetes de distinta entrada quieran ir al mismo puerto de salida.

[img]

Gestión de los paquetes encolados:

- Congestión y encolado de paquetes

- Puede haber congestion tanto en el input port, como en el switch fabric como en el output port. El encolado de paquetes puede ocurrir unicamente en el input y output port.
- Si un paquete no entra, se descarta.
- **RED:** se eligen algunos paquetes al azar y se descartan.
- Esto trae muchos problemas para TCP.

Se pueden formar colas de paquetes tanto en los puertos de entrada como en los de salida. La longitud de las colas dependerá de la velocidad del switch, del tráfico y de la velocidad de la linea. El problema es que a medida que se encolen más paquetes, mayor la chance de que se pierdan paquetes si se llena la memoria del router.

El procesador toma de a un paquete, en la medida que la cola se llena, si llegamos al limite del buffer $\Rightarrow$ empieza el desborde y no se pueden guardar mas paquetes.

- Tecnica que se llama RED: Random Early Detection la idea es que cuando el buffer comienza a llenarse(mas del 70%), se elige al azar que paquetes se descartan. A medida que se llene mas, descarta a una taza mayor. La ventaja es que lo mas probable es que no le pegue a los paquetes del mismo flujo. Esto logra que los flujos cuando detecten los ACKs repetidos disminuyan la velocidad de emisión de paquetes $\Rightarrow$ el trafico que pasa por el enlace disminuye. Si dejamos que se llene el buffer, todos los paquetes que lleguen se pierden $\Rightarrow$ timeout. Este metodo es lento, porque hasta que se enteran de bajar la taza de velocidad pasa un tiempo y mientras tanto se siguen descartando paquetes.
- Cola de prioridad: los paquetes entrantes son clasificados en clases de prioridad a medida que llegan a la cola, entonces hay una cola prioritaria y una no prioritaria. Atiendo la cola prioritaria siempre que haya paquetes, cuando esta vacia atiendo la no prioritaria. Podría generar problemas si nunca se vacía la cola prioritaria porque no atiendo a los no prioritarios $\Rightarrow$ no pasa, porque el flujo de paquetes prioritarios es muy inferior al no prioritario. Dentro de una misma prioridad, se usa FIFO.
- WFQ(Weighted Fair Queuing): los paquetes se dividen en colas de prioridad, pero se va alternando el servicio entre las colas. Así ningun paquete queda demasiado tiempo esperando en una cola. Hay una jerarquia de prioridad, a la mas prioritaria se la atiende el 50% del tiempo a la segunda un 30% y a la tercera un 20%. Asegura que todo el trafico sea atendido, por mas de que haya mucho trafico en la

mas prioritaria siempre vamos a atender a todas. Este esquema hace que todo el trafico pase.

Datagramas: nivel IP(Internet Protocol)

Segmento: nivel transporte. En la capa de red los paquetes son datagramas.

Un datagrama contiene:

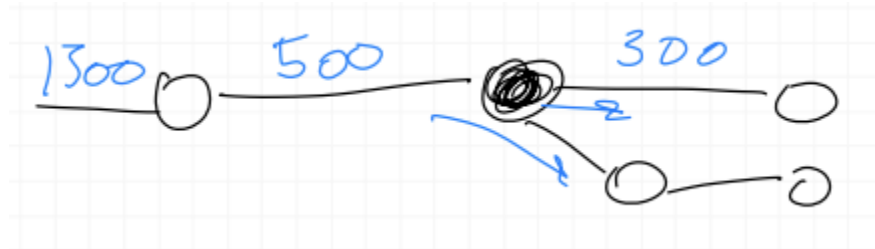
1. Version(IPv4, IPv6)
2. Direcciones: Origen, destino
3. Protocolo
4. Verificar(Checksum)
5. Robustez: TTL(*)
6. Tamaño del encabezado
7. Fragmentación: ID, frag-ID(**)
8. Datos

(*)Campo TTL(Time To Live): sabemos que las tablas de ruteo podrían estar mal, provocando que un paquete se quede loopeando entre distintos routers $\Rightarrow$ cada vez que pasamos por un router decrementamos el campo TTL en uno $\Rightarrow$ cuando se haga cero, descartamos el paquete.

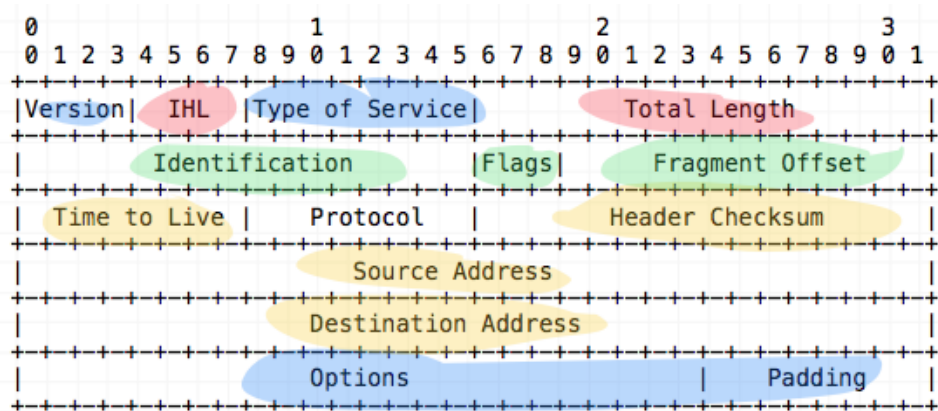
(**)Fragmentación: Hay tecnologías que usan tamaños de paquetes distintos, que pasa entonces si llega un paquete de $3N$ bytes al router pero tiene que enviar un paquete de N bytes? Se fragmenta, esta fragmentación esta dada por características de la capa fisica, por lo que existen campos que permiten hacer la fragmentación. Para fragmentar necesitamos: identificar el paquete, identificar los fragmentos a generar. Quien rearma el paquete original? Lo rearma el host de destino en la capa de Red, porque si lo rearmamos en el siguiente router puede pasar que se tenga que volver a fragmentar(es inútil volver a armar el paquete para desarmarlo de nuevo), ademas los fragmentos podrían seguir caminos diferentes.

La máxima cantidad de data que puede llevar un enlace se denomina MTU (maximum transmission unit). Cada datagrama IP se encapsula dentro de frame de capa de enlace, para se transportado de un router a otro; entonces el MTU limita mucho el

tamaño del datagram IP, el problema es que cada enlace puede usar un protocolo de enlace distinto y estos protocolos pueden tener distintos MTU's. Por esto surge la fragmentacion, las porciones de datagramas se llaman fragmentos.



Encabezado IP(v4)

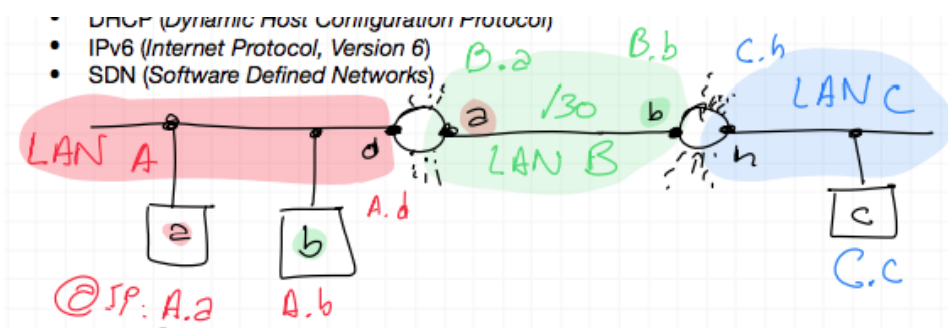


- Version: a partir de este número, el router puede determinar cómo debe interpretar los siguientes datos del datagrama.
- IHL: Header Length
- Total Length: Longitud total(incluye el encabezado)
- Con estos dos podemos saber en donde comienzan los datos(los datos estan despues del padding).
- Type of service: se solía usar para determina que tipo de servicio requería el paquete, ej: camino con baja tasa de perdida, caminos de alta capacidad, de bajo delay, etc. Hoy en día se utiliza para otras cosas, pero no se utiliza demasiado.

- Los campos relacionados con la fragmentación son los que están en verde. Todos los fragmentos de un datagrama van a tener el mismo ID (Identification), los flags: More Fragments si está en 1 significa que hay más fragmentos y si está en 0 significa que es el último. Fragment Offset: dice en dónde empiezan los datos y hay que alinearlo en double words, el primer fragmento tendría un fragment offset de 0 (si suponemos que este fragmento llevaba 2 double words), el segundo tendría un fragment offset de 2, etc.
- Time-to-live: para asegurar que los datagramas no queden circulando para siempre en la red. Si el TTL llega a 0, el router debe droppear el datagrama.
- Protocol: en general se usa cuando el datagrama llega al destino final. Indica el protocolo de capa de transporte al que se debe pasar el datagrama. El número de protocolo es el glue entre la capa de red y de transporte, mientras que el número de puerto es el de la capa de transporte y la de aplicación.
- Header checksum: para detectar errores, igual que en la capa de transporte.
- Source and destination IP addresses.
- Options: El encabezado puede tener opciones, opciones diversas, no se suelen usar.

Enrutamiento sin clases: Subredes

Tenemos una red local y en esa red conectados un Host "a" y un Host "b" que se conectan a través de un router que tiene una conexión IP "d" en la red local y un IP "a" en otra red.



Lo que vemos es que la letra "a" está repetida, por lo que pensaríamos que ambas direcciones son idénticas. Pero no son idénticas, porque lo marcado en rojo es una red

LAN llamada "A", lo marcado en verde es una red LAN llamada "B" y lo marcado en azul es una red LAN llamada "C".

Dijimos que un Host conectado a internet tiene una direccion global, compuesta por: el prefijo de red+ el prefijo de host Host a: @IP:A.a , Host b: @IP:A.b, la interfaz del router en la red LAN A se va a llamar A.d y en la otra red se llama B.a , Host c: @IP:C.c.

Si uno hace: A.a& mascara $\Rightarrow$ Red(A.a) = LAN A. El router tiene muchas interfaces con muchas redes(salvo el router wifi).

El router hace lo siguiente $157.92.58.2 \& /16 \Rightarrow 157.92.0.0$ (puede usar mascaras distintas), el router va tener que ir aplicando todas las mascaras. Pero en la unica mascara que aplica en la que encuentre un prefijo que se encuentra en su tabla de ruteo es con la mascara/16. Cuando encuentra esa direccion, el router lo saca por la salida que corresponde. Cuando llega al router de destino y aplica la mascara, la que va a ser significativa va a ser la mascara/24, ya que se trata de una IP clase B, $157.92.58.2 \& /24 \Rightarrow 157.92.\underline{58}.0$ esto es una subred.

La mascara tiene sentido localmente.

Supongamos que hacemos $157.92.58.1100\ 0000 \rightarrow /26$, De esta manera estamos armando 4 subredes, ya que tenemos 2 bits y se pueden hacer 4 combinaciones: 00, 01, 10, 11. Si esto fuese una clase C, la subred que empieza en 00 y 11 se podria confundir con la primer direccion y la ultima. Por lo que solo podriamos armar 2 subredes. Como en este caso se trata de una clase B, se pueden usar las 4 subredes.

Cuando un host envía un datagrama a la dirección 255.255.255.255, el mensaje va a ser entregado a todos los hosts de la misma subred (broadcast address).

La mascara es local, un router recibe un paquete y empieza a probar con las distintas mascaras que tiene. Toma una mascara(la mas restrictiva = la mas larga), y ve si ese prefijo coincide con alguna de las entradas en su tabla. Si no le aplica la siguiente mascara y vuelve a buscar y sigue asi con todas. Cuando encuentra una entrada lo manda, si no encuentra ninguna usa la salida por defecto, y si no tuviera una salida por defecto tiene que descartar el paquete.

Cuando un router ajeno a la organización transmite un datagrama cuya dirección pertenece a la organización, solo necesita tener en cuenta los primeros x bits, así se reduce el tamaño de las tablas de forwarding. Los últimos bits solo importan dentro de la organización.

- Cada host tiene una dirección IP.
- Diferentes tipos de hosts:
 - Computadora: Una sólo dirección IP.
 - Routers: Al menos dos direcciones IP.
- Los routers no dejan pasar los msjs de 255.255.255.255.
- Se pierde la primera y la última subred (`0` y `255`).

DHCP(Dynamic Host Configuration Protocol)

Configuración mínima(relacionado a la capa de red):

Un host tiene:

- @IP.
- Máscara. Con la mascara podemos conocer:
 - @Red.
 - @Host.
 - @Broadcast.
- Tablas de ruteo(Default Gateway)

Puerto de origen me lo da el sisop y el puerto destino esta ligado a la aplicacion.

Necesito(relacionado al nombre):

- DNS primario, secundario.
- Dominio
- Nombre del host

(`www.fi.uba.ar` === `www.` host, `fi.uba.ar` dominio).

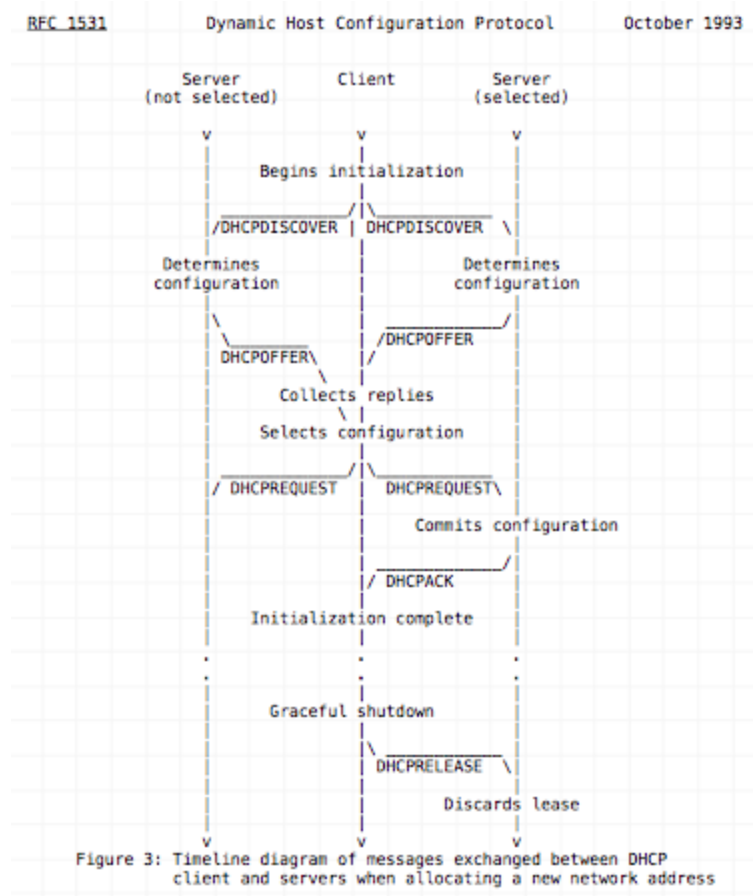
Relacionado a la capa fisica:

- MTU(maximum transfert unit)

Las direcciones IP se manejan bajo la autorización de Internet Corporation for Assigned Names and Numbers (ICANN). Una vez que el host tiene su bloque de direcciones, puede asignar IP individuales al router y host. Tipicamente un administrado de sistema

configura manual o remotamente la IP en el router. En el host se puede configurar manual o más comun es mediante el Dynamic Host Configuration Protocol (DHCP). DHCP permite que un host obtenga una dirección IP automáticamente. Además, permite que el host sepa su servidor DNS local, su máscara de subred, la dirección del primer router. DHCP es un protocolo de tipo cliente-servidor

Todo lo que hace DHCP es dar la configuración, alguno de estos son opcionales, pero la mayoría de las computadoras tienen estas configuraciones. Estas cosas son las que configura DHCP, pero como lo hace?



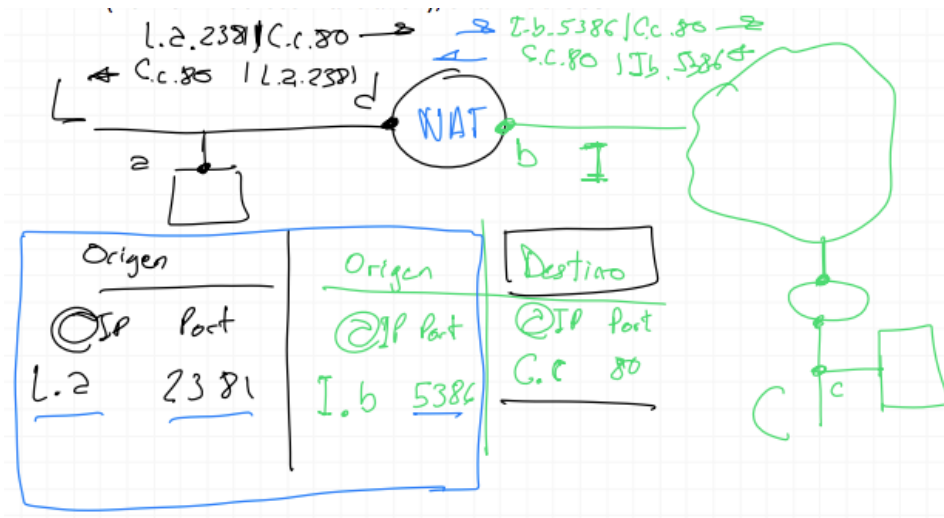
En general, hay una cuestión de redundancia, en una red por norma tiene que haber un DNS primario y uno secundario, aunque pueden haber mas secundarios de backup. Lo mismo pasa con DHCP, si se cae el servidor de DHCP es un problema por lo que suele haber varios servidor DHCP que deben ponerse de acuerdo entre ellos.

En la figura el cliente esta en el medio y a los costados dos servidores. Cuando el cliente manda un mensaje de DHCP Discover, lo escuchan todos los servidores de

DHCP y cada servidor responde ofreciendo su servicio. El cliente elige que configuración quiere, inicialmente el cliente no sabe quien es el servidor DHCP entonces los mensajes los manda a la dirección de broadcast. Como cada host esta configurado para recibir paquetes dirigidos a su dirección IP o a la dirección de broadcast de la red, lo va a escuchar. En algún momento el host decide la configuration que va a tomar y manda un DHCP Request, ese paquete dice la dirección IP de que servidor a seleccionado. Pero ese paquete tambien tiene que ir a la dirección del broadcast, porque como hay varios servidores de DHCP que estan contestando, todos tienen que estar al tanto que selecciono la oferta de algún otro, porque sino se quedarían esperando. Ya que si te ofrezco una dirección IP y vos no la usas, no se la puedo ofrecer a otro. El servidor aceptado, le va a asignar al cliente la dirección IP ofrecida con el paquete DHCP Pack. A partir de ese momento el cliente tiene una dirección para hablar en la red por lo que no necesita utilizar el braodcast. El cliente la va a usar una cantidad de tiempo, aunque se va renovando de manera automatica. Cuando voy a apagar mi computadora le aviso al servidor que no voy a necesitar esa dirección haciendo un DHCP Release, con lo que se devuelve la dirección IP.

NAT(Network Address Translation), una middlebox

Tenemos un router que tiene una dirección local d, si bien son conexiones wireless, la vamos a simbolizar con un Host, en este caso el host a. La red se llama L, la idea es que, al ser IPs privadas, para sailr al mundo de internet es necesario que haya una red I, pero nosotros queremos acceder a un host c en un red C. Tendríamos un paquete IP para acceder a un servidor web cuyo IP destino y puerto sea @IP C.c, Port 80 y del origen: @IP I.b, Port 5386. Esto es lo que haríamos si estuviésemos parados en el router, pero estamos parados en el host a, cuando mande el paquete va a poner como IP origen y puerto: @IP L.a, Port 2381 y el destino:@IP C.c, Port 80.



Ahora si yo me pongo a mirar en el cable verde que sale del router, el paquete va a tener la dirección de origen y destino que primero describimos. Por eso el router se llama Network Address Translation, el router es el que funciona de NAT, y para hacer el NAT, la información que necesita es la marcada en azul. Entonces NAT tiene que guardar una tabla en donde sepa que IP de origen y que puerto esta conectado a que puerto de salida (los datos subrayados en azul).

IPv6 (Internet Protocol, Version 6)

Porque se tuvo que hacer una nueva version de IP? Motivación principal: **falta de direcciones**, hay una parte de las direcciones que no se pueden usar por subnetting.

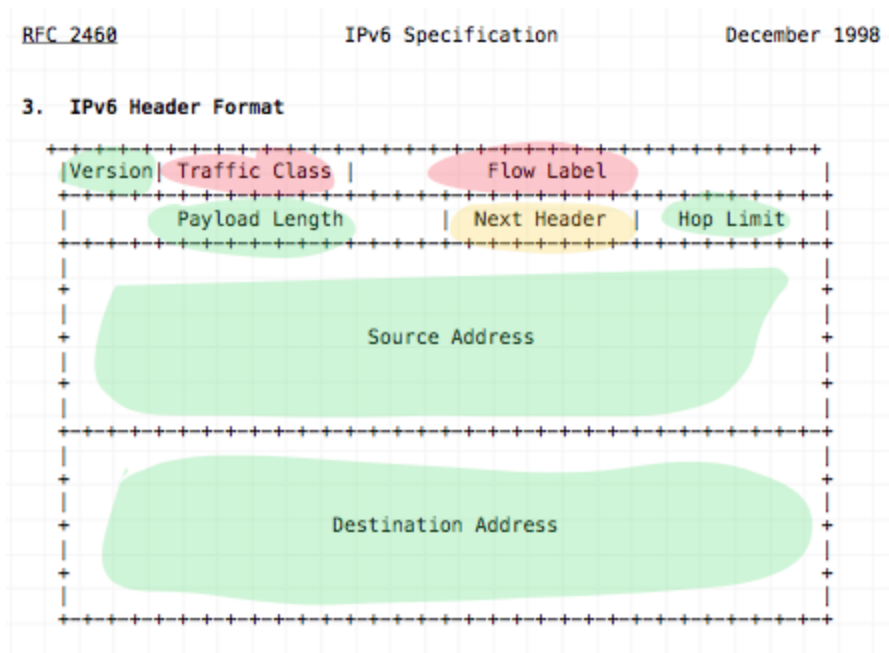
Una de las cosas que se hizo fue que había partes de los encabezados que típicamente no se usaban como el de fragmentación del datagrama ya que no siempre se fragmenta sino que a veces sucede. Otros campos que no son tan importantes $\Rightarrow$ se cambio a un encabezado minimalista y si quiero un modulo de fragmentación le agrego ese modulo, se agrega un modulo unicamente cuando se necesita. Aprovechando que cuando hay fragmentación hay un problema: cada vez que un router hace una fragmentación, los cálculos son lentos por lo tanto no es una actividad buena para el router $\Rightarrow$ le lleva tiempo y hace mas lenta las conexiones. Se decidió que la fragmentación no ocurra en los routers, puede haber a nivel IP pero unicamente en los puntos extremos (endhosts).

Por lo que se agrego **encabezados modulares**, **fragmentación prohibida en los routers** (solo los extremos), la **autoconfiguración**, **tratamiento diferenciado de flujos** (no se usa demasiado) y **no checksum**. Cambios respecto a la versión anterior:

más direcciones disponibles y header fijo de 40 bytes, permite que se procesen más rápido los datagramas en el router.

Hacer el forwarding de los paquetes de IPv6 es muy costoso en los routers, al menos que no haya encabezados opcionales → genera que no se usen los encabezados opcionales.

Encabezado IPv6



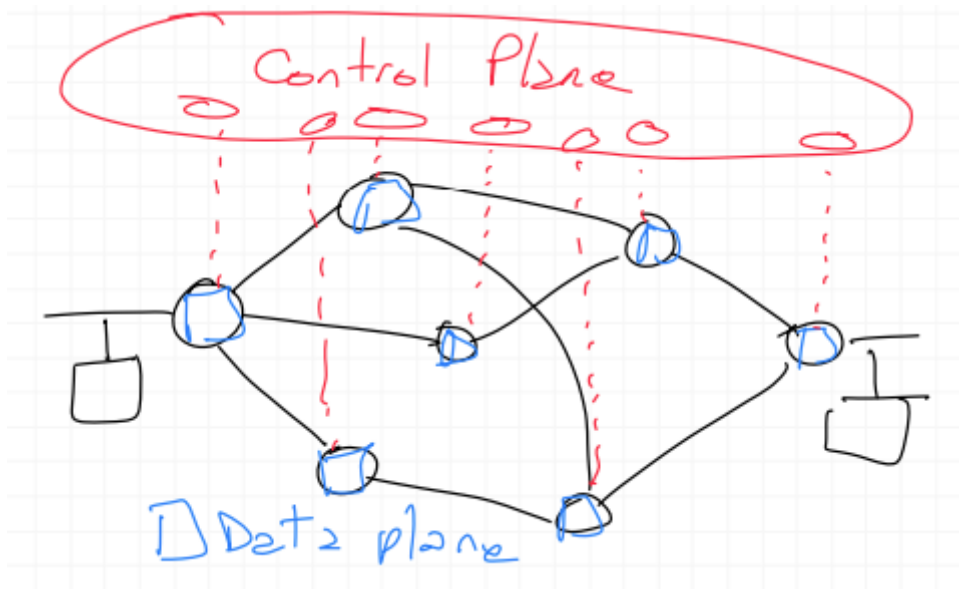
Empezamos con 4 bits para la version, luego hay dos campos Traffic Class y Flow Label que están dedicados a hacer mejor tagging de los paquetes. Traffic Class: como el campo TOS de IPv4, se puede usar para dar prioridad a ciertos datagramas dentro de un flujo, no son campos muy útiles los marcados en rojo. La idea es poner un Flow Label a cada flujo, si yo genero un flujo, genero una etiqueta y utilizo en todos los paquetes que mando la misma etiqueta. Sin embargo los routers no necesariamente se basan en el Flow Label por si alguien maliciosamente escribe un Flow Label → por seguridad.

Cuando construimos un paquete tenemos el header principal y luego otros headers (por ejemplo header de TCP o UDP) y finalmente está el payload que son los datos. Por lo que el campo Payload Length refiere a la longitud de los datos. Luego el campo Next Header indica que tipo de encabezado es el que viene (por ejemplo TCP o UDP);

fragmentation header; routing header). Cuando el Next Header es capa de transporte es útil y sino no lo es. El campo Hop Limit es el equivalente al Time To Live, sería la cantidad de hops(routers) que atraviesa, se va decrementando en uno cada vez que un router lo forwardea. Finalmente tenemos la dirección de origen y la de destino.

Cómo se lleva a cabo la transición de IPv4 a IPv6? via tunneling. La idea básicamente es encapsular la información del datagrama IPv6 en el payload de un datagrama IPv4, así dentro del router pueden viajar como IPv4, pero en los extremos se desmantela como IPv6 (se identifica que debe hacerse esto a partir del protocol number field).

SDN (Software Defined Networks)



La operación de forwarding es la que se encuentra en el Data Plane, este si o si funciona en cada router. El Data Plane es responsable de este routing, esta la tabla de routeo y viene el paquete, tiene que hacer el routing pero una vez que eligió el puerto de salida hace de forwarding. El Control Plane es quien escribe las tablas de routeo y puede estar en cada router o bien puede haber un control centralizado que lo coordine. SDN se encarga de hacer el control centralizado, no necesariamente debe ser centralizado sino que se pueden tener varias areas que se comuniquen entre ellos y coordinen esto. Adentro de los routers esta solamente el hardware que permite hacer muy rapidamente el matching de cosas y decidir por donde salir.

OpenFlow:

Ingress Port	Ether source	Ether dst	Ether type	VLAN id	VLAN priority	IP src	IP dst	IP proto	IP ToS bits	TCP/UDP src port	TCP/UDP dst port
--------------	--------------	-----------	------------	---------	---------------	--------	--------	----------	-------------	------------------	------------------

Table 2: Fields from packets used to match against flow entries.



Podría tomar decisiones de que hacer con el paquete mirando estos campos. Esta información es altamente utilizada por los middleboxes.

Libro:

Un controlador remoto distribuye las tablas de forwarding para cada router → SDN (software-defined networking), se llama así porque el controlador que interactúa con los routers está implementado en software en data centers remotos.

OpenFlow Protocol

Es un protocolo que opera entre un controlador SDN y un switch controlado por SDN.

Entre los mensajes que intercambia están los siguientes:

Configuración. Consultar y setear parámetros de configuración de un switch.

Modify-State. Agregar o eliminar entradas de la tabla de flujo o setear propiedades a los puertos.

Read-State. Estadísticas y contadores de la tabla de flujo y puertos.

Send-Packet. Envía un paquete específico a un puerto específico.

Flow-Removed. Avisa que una entrada de la tabla fue removida.

Port-Status. Avisa cambios de estado en los puertos.

Packet-in. Para mandar paquetes al controlador.

Middleboxes:

- NAT
- Firewall

- DPI (IDS)

Plano de Control - Control Plane

Controlar como se van a enviar los datos en la red.

El Data Plane es el que se encarga de hacer el forwarding: el reenvio de los paquetes que llegan y existe un procedimiento que hace el reenvio. Hablamos de Routing, cuando hay que consultar la tabla que debia matchear con el prefijo mas largo. Cuando matchea esa es la entrada valida y me fijo por que salida va.

Cuando pasa el primer paquete del flujo, el router hace el ruteo(consulta a la tabla), pero una vez que ya sabe por que salida tiene que salir, genera una funcion de hash con esos campos y anota el puerto de salida en con esa funcion de hash → si el router cambia su tabla de ruteo, hay que cambiarla.

El control plane lo que hace es encargarse de la escritura de la tabla de ruteo, aca es donde funcionan los protocolos de ruteo. El protocolo de ruteo funciona estrictamente en la capa de aplicacion. La tabla de ruteo opera directamente en la capa de red.

Distribuido vs. Centralizado

Tenemos una Red con caminos alternativos, cada router debe tomar decisiones relacionadas con el Data Plane (por donde envia los paquetes, ya sea el ruteo o el forwarding). Sin embargo para hacer eso, cada router tiene sus tablas de ruteo, la diferencia en el control plane es quien escribe las tablas? Existen dos versiones:

1. Hay un agente que funciona en cada router - version distribuida.
2. Todos los routers hablan con un controlador central que es quien escribe las tablas de ruteo.

Cuando uno discute que es lo que pasa en Internet, todo internet no podría funcionar con un ente centralizado. Esta idea de control centralizado puede funcionar a nivel FIUBA, pero no en todo el Internet. En la mayoría de Internet funciona el control distribuido, los algoritmos que escriben las tablas de ruteo funcionan en cada router en particular.

Dinámico vs. Estático

A nivel routing tenemos estatico o dinamico. Un caso estatico serian: la tabla de ruteo de la computadora, el NAT. Es decir tenemos una computadora con una dirección IP privada, y tenemos un router conectado a nuestro host. Este router del lado conectado a internet tiene una dirección publica, en el router es donde funciona NAT. El router necesita una dirección estatica ya que cuando uno esta conectado a una red, ya sabe como enviar datos. El problema es cuando la computadora tiene que enviar algo hacia afuera, se lo va a enviar al router, y el router para enviar hacia afuera se lo manda a la puerta que esta conectada la dirección publica(Internet), eso es una dirección fija, es el **default gateway**. Eso es el ruteo estatico, es una tabla de ruteo estatica, que no cambia en una sesión de trabajo.

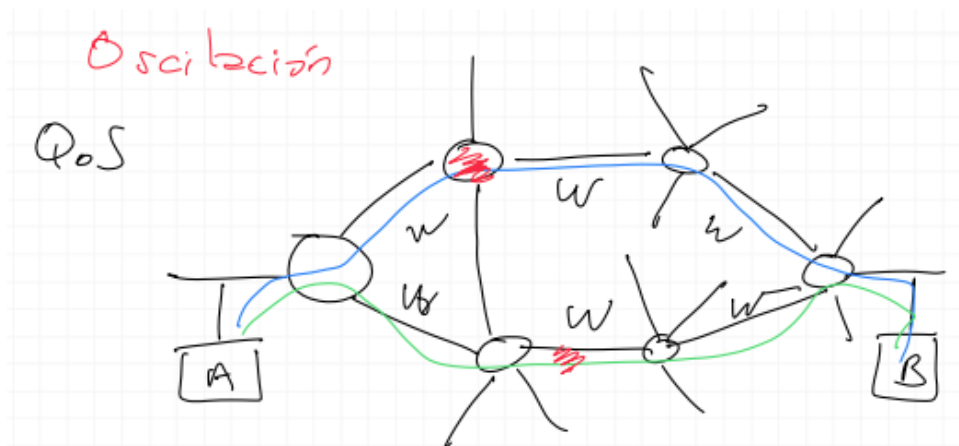
Ruta por defecto:

- Estática.
- Todo el tráfico que no matchee con ninguna entrada, va a ir por ahí.
- Cualquier dirección matchea, pero primero hay que chequear las demas.

Dinamico, en cambio es cuando hay un protocolo de ruteo, hay que modificar las salidas de los routers con distintas direcciones. Hay necesidad de que los protocolos de ruteo sean dinamicos ya que pueden haber rutas nuevas o otras que se cierran, etc.

Dependientes, o no, del tráfico.

Tenemos dos host A y B que se conectan a priori por dos rutas equivalentes. Uno podría enviar por el flujo azul, donde los routers del camino estan inmersos en la internet por lo que pueden tener flujos de otros lados. El router en rojo se congestiona, entonces podríamos elegir el otro camino, en ese momento el camino verde tambien se congestiona. De esta manera terminamos oscilando entre los dos caminos. Los protocolos de ruteo que intentan resolver este problema a traves del ruteo dinamico(cambiar la ruta en función del trafico) tienen este problema: oscilación. Si nuestros protocolos de ruteo son dependientes del trafico $\Rightarrow$ puede haber oscilación.



Si no fueran dependientes del tráfico podemos elegir el camino según la calidad de servicio (Quality of Service), es decir, quiero una ruta con tal características. La calidad del servicio depende del parámetro que uno quiera medir. El tráfico de datos, tiene una estadística no acotada, por lo que no vamos a medir lo mismo en distintos intervalos de tiempos. Esto no ocurre en la Internet porque es muy complicado establecer un protocolo que se ocupe de otra cosa que no sea encontrar un camino que exista. Una forma de establecer la calidad de servicio, es estableciendo pesos en los caminos, sumando los pesos podemos determinar la calidad de un camino respecto del otro $\Rightarrow$ problema de la oscilación si hacemos variar los pesos con el tráfico.

Organización de Internet

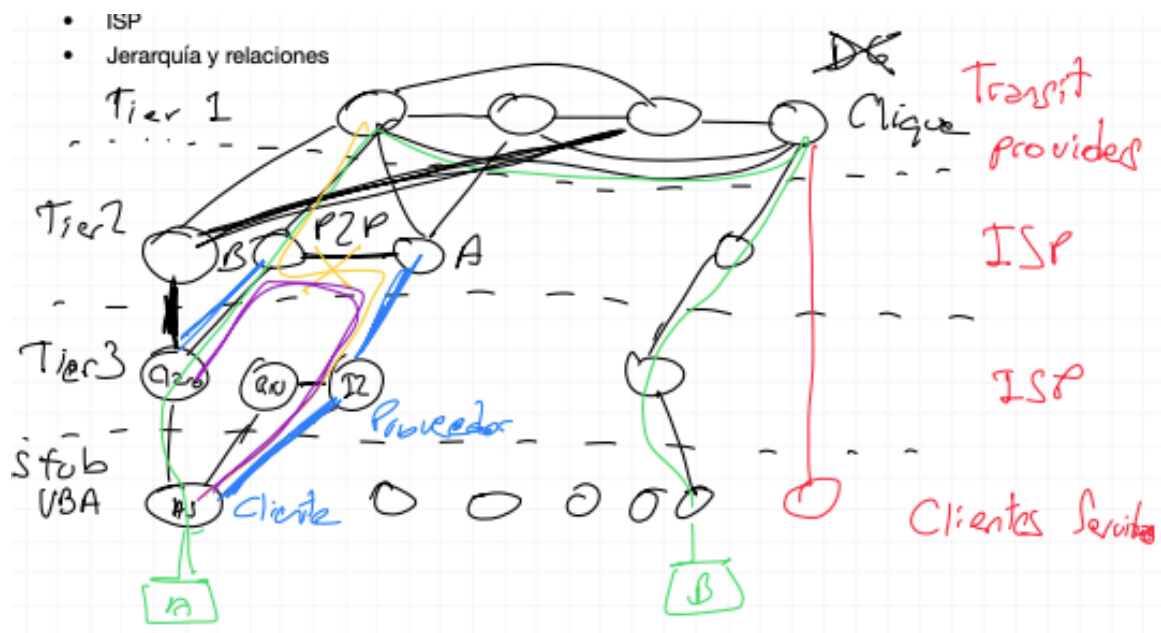
- ASes
- ISP (Internet Service Provider)
- Jerarquía y relaciones

Internet está organizado en **sistemas autónomos**. Cada sistema autónomo (SA) consiste de un grupo de routers que están bajo el mismo control administrativo. Los routers que están dentro de un mismo SA, corren el mismo algoritmo de routing y tienen información sobre los demás routers.

Se organiza de esta manera ya que los SA, hacia el resto de Internet se lo ve como un conjunto que se comunica con el resto de Internet. Un ISP es un SA y hay otros tipos de SA. Vamos a ver una clasificación:

Tenemos el SA de la UBA que tiene como proveedores: Claro, Internet2, RIU (Red inter-universitaria). Para ser un SA uno tiene que tener al menos 2 proveedores distintos.

Estos proveedores pueden tener otros proveedores, por ejemplo RIU esta conectado a I2, estas conexiones se llaman P2P(peer to peer), porque estan en el mismo nivel. Estos proveedores se pueden conectar a otros proveedores en otro nivel y asi. Todos los tiers salvo el 1 van a tener un default gateway en algun lado, pero los Tier 1 no lo tienen, y deben conocer todos los destinos. Todos los Tier 1 deben estar conectados con todos.



- Tienen ASN (AS Number). Lo entrega el IANA.
 - Tier 1 -> Tier 2 -> Tier 3 -> Stub.
- Conexiones peer to peer (mismo nivel).
- Conexiones hacia a arriba: cliente-proveedor.
- Los que están en el Tier 1 no pueden usar rutas por defecto(default gateways).
 - Deben ser rutas exactas.
- Todos los Tier 1 deben estar conectados con todos(Clique).

Nuevas jerarquías

Si nosotros somos el host B y queremos conectarnos con la pagina de la UBA, la ruta va a ser uphill, luego se mueve en un solo hop al router de salida y luego se hace el downhill. Este era el concepto inicial de Internet donde Tier 1 son los Transit providers,

los Tiers 2 y 3 son ISPs y los stubs son los clientes/servidores. Pero en el desarrollo de internet se permitió que el tier 1 de servicio a un stub directamente $\Rightarrow$ desarmo la estructura. Además se dejó de respetar que los tiers 1 estén conectados todos con todos.

- Del Tier 1 directo al Cliente para ganar más dinero.
- 82 niveles de jerarquía.
- Sistemas autónomos.
- Se mantiene la idea de mundo pequeño, ya que la máxima distancia entre nodos es de 16 aproximadamente.

Hay dos visiones: una de lo que ocurre dentro del SA y otra de lo que ocurre por fuera. Esta visión se ve reflejada en los protocolos de ruteo, ya que dijimos que no podemos tener un protocolo de ruteo centralizado para todo internet. Pero eventualmente para alguno de estos SA podríamos tener un protocolo de ruteo centralizado. Hay otra propiedad importante de los SA: hay algunos de los SAs que están bastante concentrados, pero en general el SA no tienen una localización geográfica clara, algunos en mayor superficie que otros.

Vamos a definir **protocolos de ruteo intra e inter**, intra: es al interior de un SA e inter: entre los SAs. El protocolo de ruteo entre los SAs es único, es el único que se utiliza en Internet ya que hace falta que sea un idioma universal en el que hablan todos. Dentro de cada SA cada uno es libre de utilizar cualquier protocolo de ruteo.

Protocolos de Ruteo

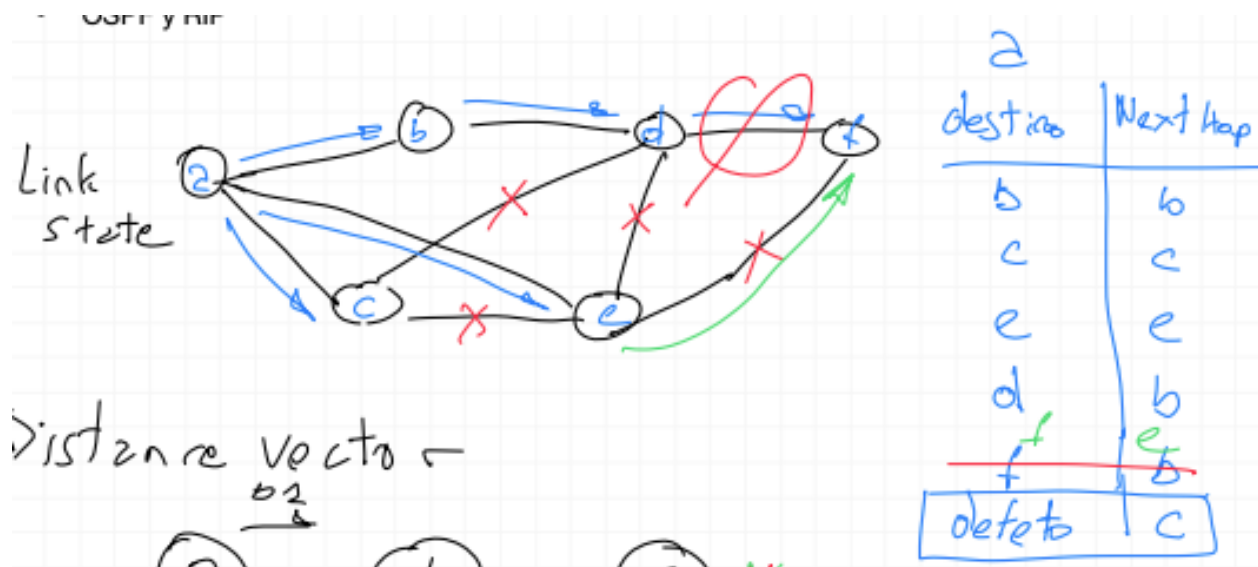
| Distintos puntos de vista

Link-state

- El protocolo para cada nodo construye un árbol.
- Es centralizado en el sentido de que todos los miembros de la red tienen información del resto de la red (Cada router sabe con quien está conectado).
- Cada router recibe información de toda la red, esto lo hace a través de un proceso llamado "flooding" en donde todos van avisando a todos lo que pasa.

- Mira el estado de sus enlaces de forma permanente, y si hay algún cambio, lo tiene que avisar.
- No es aplicable a todo internet, no escala.
 - Porque al ser muy alta la probabilidad de que se caiga un enlace y por cada enlace que se cae se lo debo informar a todo el mundo $\Rightarrow$ estaríamos enviando todo el tiempo información sobre como esta la topologia de internet en vez de enviar información importante en la red.
 - Por otro lado necesitamos que cada router tenga memoria suficiente para guardar la topologia y capacidad e calculo para hacer el calculo.
- Si el sistema es muy grande, tampoco escala.
 - Para cada update tengo que hacer el algoritmo, consumiendo mucho CPU.
- Se puede utilizar como protocolo intra-sistema autonomo

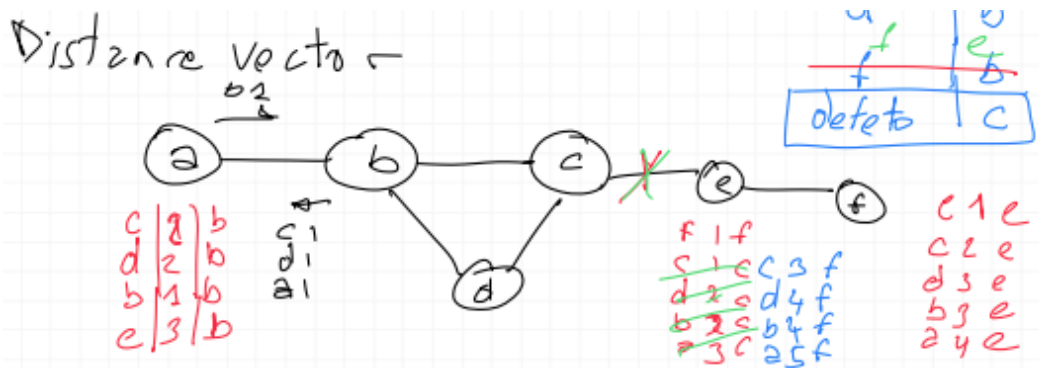
En este tipo de algoritmo, la topología de red y todos los costos de enlace son conocidos. Esto se logra haciendo que todos los nodos broadcasteen paquetes linkstate a todos los demás nodos de la red. Los algoritmos que se usan son el de Dijkstra o el de Prim. El algoritmo de Dijkstra es iterativo, tiene la particularidad que después de la k-ésima iteración, los caminos menos costosos van a ser conocidos por k nodos destino, y entre esos caminos menos costosos esos k caminos van a tener los k costos más pequeños.



Distance-vector

- Cada nodo le transmite a sus vecinos despues de un tiempo su tabla de ruteo que consiste en un vector, donde dice a qué distancia esta de cada router.
- Para cada destino, a qué distancia está, respecto de ese router en particular.
- Cuando el router X le transmite su tabla de ruteo a Y no le transmite las entradas que aprendió a traves de Y.
- En redes mas grandes no hay manera de evitar bucles a una cierta distancia, a medida que la red se agranda se pueden dar bucles a mayor distancia.
- Si aplicamos distance-vector a todo internet, la convergencia va a tardar en alcanzarse.

Es iterativo, asincrónico y distribuido. Es distribuido en el sentido que cada nodo recibe información sobre sus vecinos directos, hace cálculos y distribuye los resultados devuelta a los vecinos. Es iterativo en el sentido de que el algoritmo continúa hasta que no hay más intercambio de información entre vecinos. Es asincrónico porque no necesita que los nodos operen lockeandose entre ellos.



OSPF (Open Shortest Path First)

- Utiliza Dijkstra
- Permite usar pesos relacionado con las capacidades de los enlaces o con las prioridades de los enlaces.
- Autenticacion: capa de seguridad
- Zonas: se crea una zona cero que se comunica con otras zonas y lo que hacen es confinar el ruteo en burbujas. De esta manera cada zona maneja sus routers y no

importa lo que ocurre dentro de cada zona. Las conexiones entre zonas son permanentes.

- Soporta multicast nativo

Es un protocolo de link-state que utiliza “flooding of link-state information” y el algoritmo de Dijkstra para el camino menos costoso. Los routers broadcastean la información a todos los demás routers, no solo a sus vecinos.

RIP

- Utiliza Distance-vector
- Autenticación

SDN(Software Defined Networks), Control Plane

- Reenvío basado en flujos
- Separación del plano de datos y de control
- Control de Red: externo a los conmutadores
- Red Programable

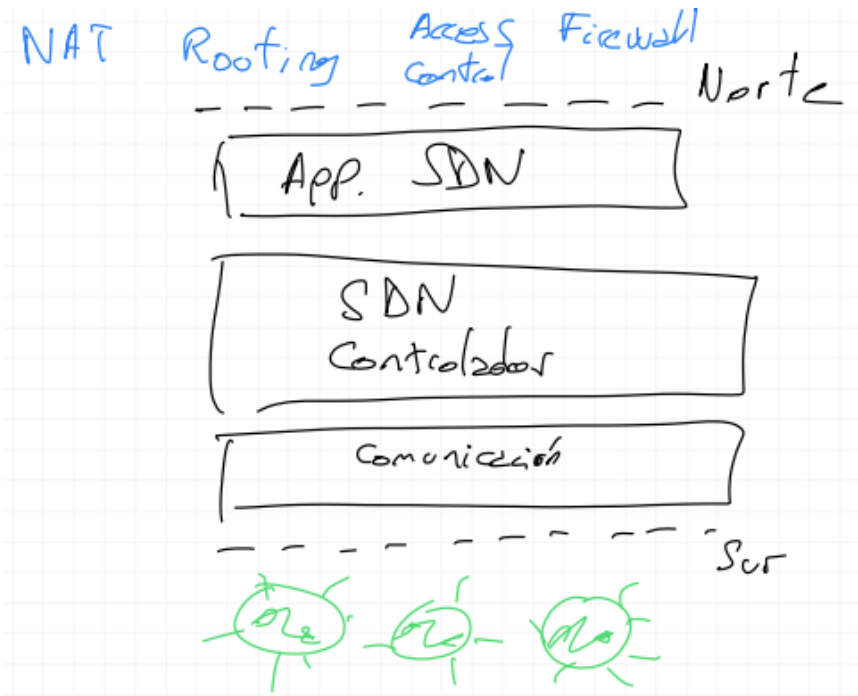
Control plane

- Devuelve tablas de reenvío, de forwarding, para cada router. `Red | Salida`.
- Hay protocolos en los que las tablas se construyen en los routers, que se comunican con el resto de los routers para lograrlo.

Lo que hace el plano de datos es mirar los paquetes que se envían y decidir según diversas cosas por qué salidas salen. En los comienzos de Internet lo que se miraba únicamente es la IP destino ya que era lo más simple, luego hubieron modificaciones como las que trajo SDN.

En el caso de SDN la idea es que el plano de control sea centralizado, por lo que hay una conexión a ese plano de control donde se realizan varias operaciones (escritura de la tabla de ruteo, etc). Todas las cuestiones de middlebox pueden estar manejadas por las SDNs. Las SDNs están basadas en los flujos, en la separación de plano de datos y plano de control. La tabla de ruteo tiene dos partes: a dónde lo quiero enviar y el puerto de salida (próximo hop).

La SDN tiene sentido dentro de un sistema autonomo porque podríamos tener un montón de dispositivos que cuando no tienen en la tabla de forwarding los datos le consulta al plano de control. El plano de control es quien escribe la ruta en todos los dispositivos a donde se tienen que enviar $\Rightarrow$ opera ademas de en la capa de red en la de enlace.



Cuando trabajamos con SDN tenemos un esquema con capas, dentro del limite Norte-Sur se encuentra el controlador que tiene: una capa de comunicación, el controlador de SDN y las aplicaciones propias de SDN. Arriba de eso el usuario implementa distintas cosas: control de acceso, firewall, NAT, etc. Hoy en dia todo esta en containers como los servidores, esta es la idea y nos da un gran flexibilidad. Por debajo de la capa sur tenemos los dispositivos (switches, routers). No es factible para todo el internet, si tenemos una organización de SAs y cada SA decide como se organiza y administra su red, no queremos que haya alguien que decida como administramos todo internet, sino que queremos que sea una federación de redes que cada una dentro de su red haga lo que quiera y para afuera convivimos con ciertas reglas. Se logra robustez, a medida que tenemos mayor redundancia $\Rightarrow$ ventaja. La SDN nos permite tener la capacidad de adaptarse rapidamente a distintas situaciones.

Comentarios generales:

- Se puede aplicar SDN a cualquier red.
- Lo aplicamos dentro de un sistema autónomo(Intra).
- Es un poco más que IGP (Internal Gateway Protocol).
- Opera con **flujos**: capa 4 para arriba (de Transporte para arriba).
 - Ocurre en la capa de transporte o aplicación.
- Intenta identificar flujos de distintas maneras.

BGP (Border Gateway Protocol)

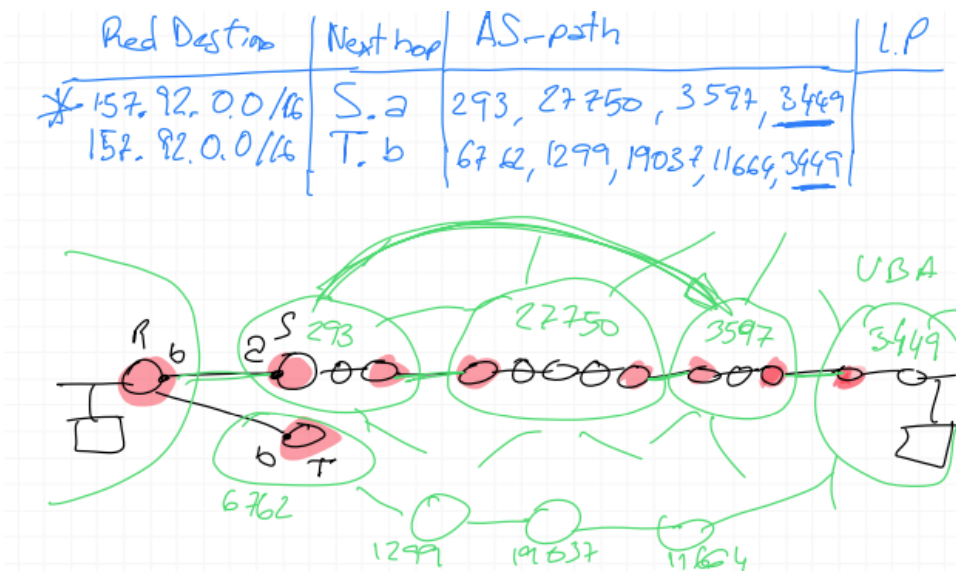
Protocolo de ruteo.

- Uso y aplicación
- Funcionamiento
- Implementación de Políticas

En Internet hay muchos sistemas autonomos, en esa red tenemos que encontrar un protocolo de ruteo que escale. Primero en vez ver Internet plana donde son solo routers, miremos una Internet que este gobernada por sistemas autonomos. En vez de halar entre redes, vamos a hablar entre SAs. Estamos interesados en conocer la ruta entre los SAs, estos se comunican entre si a traves de los routers borde o de frontera(routers rojos).

BGP es un protocolo donde la salida de cada SA habla con sus pares del otro lado de la frontera. Un SA puede tener multiples conexiones hacia otros lados. Los SAs tienen numeros, con los que se los identifican, estos numeros son otorgados por IANA. En todos los routers bordes estamos obligados a correr BGP. BGP tiene sentido entre SAs, esto es lo que el libro llama E(xternal)BGP.

| Con 16 bits podemos tener hasta 65 535 sistemas autonomos.



Hay mas prefijos que SAs, ya que cada prefijo es una red que administra no necesariamente tiene una red, puede tener muchas redes, entonces por cada red va a tener un prefijo. Por cada prefijo tengo que tener una linea en mi tabla de ruteo. BGP es distance-vector pero sin usar Dijkstra.

Armado de tablas de ruteo:

- Red Destino. Ej: 157.92.0.0/16
- Next Hop. Ej: S.a. (router: S, entrada: a)
- AS path. Ej: 293, 27750, 3597, 3449
 - Esto lo hace para no tener que usar Dijkstra y tener que hacer los cálculos constantemente.
 - Significa que el prefijo 157.92.0.0/16 esta originado en el SA 3449, quien es dueño del prefijo.
- X: Local preferences.

En este caso es facil salvar los bucles porque tenemos el AS path completo, sabemos por todos los SAs que atraviesa. De esta manera BGP se asegura de tener entradas en la tabla de ruteo que no tengan bucles y por eso tiene una convergencia muy rapida.

Lo que hemos visto es que por cada destino guardan una sola entrada en la tabla de ruteo, entonces si se corta una conexión el router va a necesitar una ruta nueva. BGP lo que hizo es tener varias rutas para un mismo prefijo. Como decido por cual ruta voy?

Menor cantidad de saltos, es la única información en la cual puedo confiar a nivel global.

Si se cortase el enlace entre el router R y S, elimino esa entrada de la tabla de ruteo y notifico a todos mis vecinos BGP (no necesariamente son vecinos conectados a través de enlaces físicos) la información de que se cayeron todas las entradas que tengo con ese vecino.

BGP puede aprender muchas rutas a un destino, elige la mejor simplemente mirando la cantidad de saltos en el AS path y eso es lo que usa cuando sale al SA. Adentro del SA tiene que retransmitir esa info hacia el protocolo de ruteo interno, por lo que esos routers rojos muchas veces tienen que correr un protocolo de ruteo interno y otro externo. El externo BGP, el interno podría ser BGP o OSPF, RIP, etc. De alguna manera debe generar esa información para que cualquiera dentro del SA pueda conocer destinos a los prefijos que quiere viajar. (Ver hot potatoe, etc del libro).

Si o si afuera de mi SA hablo BGP, por lo tanto tengo que escuchar lo que dicen mis vecinos y aprender esas rutas y elegir la mejor contando la cantidad de hops a nivel SA (no implica que sea la más corta).

BGP se diferencia de los otros protocolos por la posibilidad de aplicar políticas. Si por ejemplo la UBA quisiese recibir tráfico más de un proveedor (11664) que del otro, como hacemos? AS Prepend: en el AS path no podemos poner un SA que no controlemos, por lo que la UBA como controla el 3449 podría anunciar en el AS path 2 o 3 veces su número de SA. De esta manera si en el primer AS path agrega su número de SA 2 o 3 veces, cuando haya que elegir entre un camino y el otro, el segundo camino va a ser el más corto. A esto se lo llama: Ingeniería de tráfico. Lo único que hace esto es decirme por donde viene el tráfico, puedo controlar la llegada de tráfico, no la salida.

Local preferences, a una ruta particular le puedo agregar un peso y esa información cuando hacemos la transferencia del protocolo de ruteo interno al externo, lo puedo usar para seleccionar cosas. Las local preferences una vez que se envían al vecino, este puede hacer lo que quiera con ellas, ignorarlas o no. Lo que no puede tocar es el AS path, puede agregar repetidas veces su número de SA pero nada más.

Implementación de políticas de ruteo en BGP:

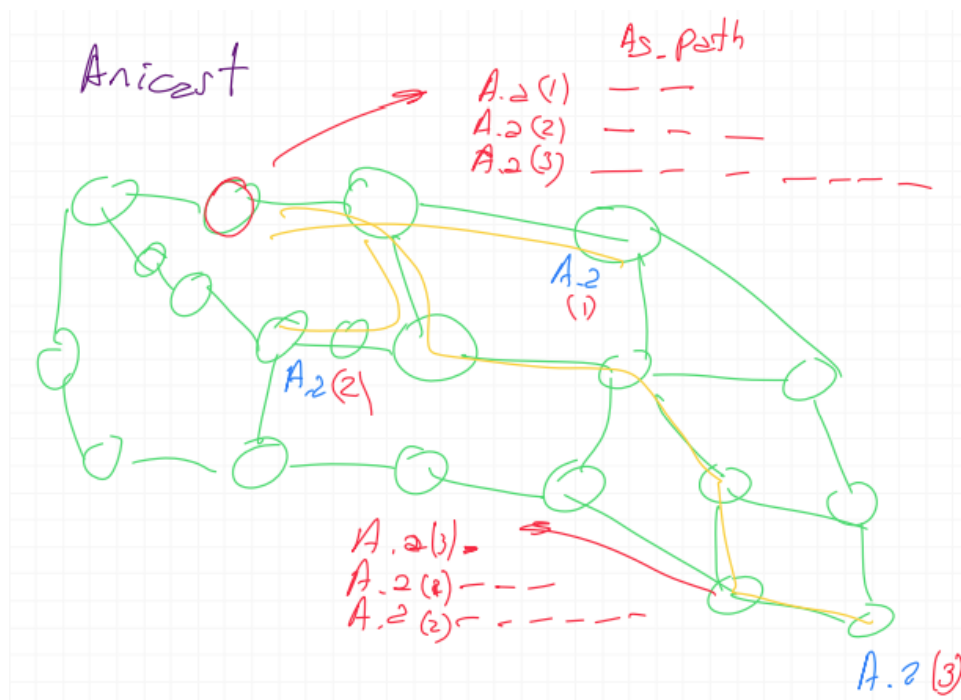
Los tráficos marcados en amarillo no pueden suceder porque estaría gastando recursos que se lo estoy dando a alguien que está en mi mismo nivel. Por lo que para que esto no suceda: lo que se aprende de los clientes se distribuye a los pares y hacia arriba;

pero lo que se aprende desde arriba solo se distribuye a mis clientes(NUNCA a mis pares). De esta manera BGP controla las politicas de trafico.

Se crea la accion MANRS(Mutually Agreed Norms for Routing Security) para que no haya secuestro de rutas(ejemplo: que por algun motivo por ejemplo I2 crea que puede llegar mas rapido a Google a traves de la UBA y se redirige todo el flujo para que pase por la UBA). Esta acción consiste en:

1. Filtrar/Validación global: no vamos a propagar todas las rutas, nos aseguramos que todas las rutas sean validas, que el SA originante del prefijo realmente sea dueño del prefijo. Solo voy a enunciar rutas de mis clientes.
2. Anti-spoofing: si un cliente mio me manda un paquete IP cuya direccion de origen no esta contenida en el espacio de direcciones que el controla, la tengo que eliminar.
3. Coordinación: Se pide un comunicación global.

Anicast:



Uno de los problemas de las tablas BGP es que el numero de entradas crece mucho. A medida que hay más redes, hay más entradas en la tabla de routeo. "De alguna manera lineal". Problema que tiene Internet, habría que cambiar BGP. [a chequear]

Libro:

Routing Among the ISPs: BGP

Cuando se routea un paquete dentro de un AS, la ruta la define el protocolo de intra-AS routing. Pero, un paquete pasa por varios ASs, para eso se necesita un protocolo de inter-AS routing. En internet, se utiliza el protocolo BGP (Border Gateway Protocol). Es el protocolo que hace de pegamento entre las miles de ISPs que tiene internet. BGP provee a cada router:

Obtener un prefijo de accesibilidad para la información de ASs vecinos. Una subred “avisa” de su existencia y BGP se asegura de que todos los demás ASs sepan sobre esta subred.

Determinar la mejor ruta a los prefijos

Para cada AS, cada router puede ser un gateway router o un internal router. Un gateway router es el que está en el “borde” de un AS y se comunica con uno o más routers de otras ASs. Un internal router se conecta únicamente con routers y hosts del mismo AS.

En BGP, pares de routers intercambian información sobre conexiones TCP semipermanentes (usando el puerto 179). Cada conexión TCP se conoce como BGP connection. Una conexión BGP que comunica dos ASs se llama external BGP (eBGP), mientras que una conexión BGP entre routers del mismo AS se conoce como internal BGP (iBGP).

Cuando un router avisa sobre un prefijo a través de la conexión BGP, incluye varios atributos BGP; en la terminología BGP a un prefijo acompañado de sus atributos se lo llama route. Los dos atributos más importantes son AS-PATH y NEXT-HOP. AS-PATH contiene la lista de ASs por los que pasó el “aviso”, se usa para detectar y prevenir loops. El NEXT-HOP es la dirección IP de la interfaz del router que comienza el ASPATH, es básicamente la dirección IP del router más cercano del AS adyacente.

El algoritmo más simple de ruteo es el hot potato routing. En este algoritmo, el camino elegido es aquel con el menor costo al router NEXT-HOP.

La idea del hot potato es, visto desde cada AS, quitarse de encima lo más rápido posible a los paquetes sin tener que preocuparse por el costo de los paquetes que ya estan fuera del AS. Se dice que es un algoritmo egoísta porque trata de reducir el costo del propio AS sin preocuparse por los costos externos.

[img]

En la práctica, el algoritmo que se usa es el de route-selection. El input de este algoritmo es un set con todas los caminos hacia el prefijo, que ya fueron aprendidos y aceptados por el router. Si hay más de un camino al mismo prefijo, se aplican reglas de eliminación:

Un camino tiene asignado un atributo llamado 'local preference' tal que se eligen los caminos con el mayor valor.

De los restantes, se seleccionan los caminos con el AS-PATH más corto.

Luego se utiliza el algoritmo de hot potato.

Si siguen quedando más de un camino, se utilizan identificados BGP.

Dato de color: además de el protocolo de ruteo inter-AS, BGP se suele usar para implementar el servicio IP-anycast (comúnmente usado en DNS). Cuando un host envía un datagrama a una dirección anycast, la infraestructura de red buscará el camino más corto hasta uno, y preferiblemente solo uno, de los equipos que aceptan datagramas dirigidos a la dirección anycast utilizada. El algoritmo de BGP provee una manera facil y natural de hacerlo.

ICMP(Internet Control Message Protocol)

- Utilidad
- IPv4: RFC 792
- IPv6: RFC4443

La utilidad de este protocolo es poder enviar mensajes de control(NO es protocolo de transporte). ICMP es un protocolo de señalización, denominado inband, porque va dentro de la misma banda donde estan los datos, que lleva informacion de la red. Los paquetes de ICMP pueden ser de informacion o de errores.

Errores:

1. Destination Unreachable → si queremos generar una conexion hacia una IP que sabemos que son clientes, no hay ningun servicio, esto nos da un Destination Unreachable. Hay distintos tipo de mensajes Destination Unreachable contemplando el tipo de respuesta(si es un host, una red, un servicio particular, etc).
2. IPv6: Packet too big
3. IPv4: Fragment needed but DF=1

4. Packet corruption: dropeo de paquete
5. Time exceeded:
 - during transit: el router ve que el time to live expiro entonces lo descarta
 - during reassembling: cuando se pierde un fragmento
6. Parameter problem(casi no se usa)

Informacion:

1. Echo
 - Request
 - Reply
2. Network information
3. Source quench: es un paquete que un router podria enviar a una direccion IP informandole que este router esta congestionado
4. Redirect: un host en una red con dos routers, se lo manda siempre a su default gateway y ese router dice que la salida es el otro. Entonces cuando el host recibe ese paquete incluye en su tabla de ruteo una ruta que va al destino que quiere ir a traves del otro.

Entonces ICMP, viaja en un paquete IP, en la parte de datos, no es un protocolo de transporte. Tenemos que enviar información que llegue al host que origino la comunicación, entonces tenemos que actuar a nivel IP, luego el host decide que hace.

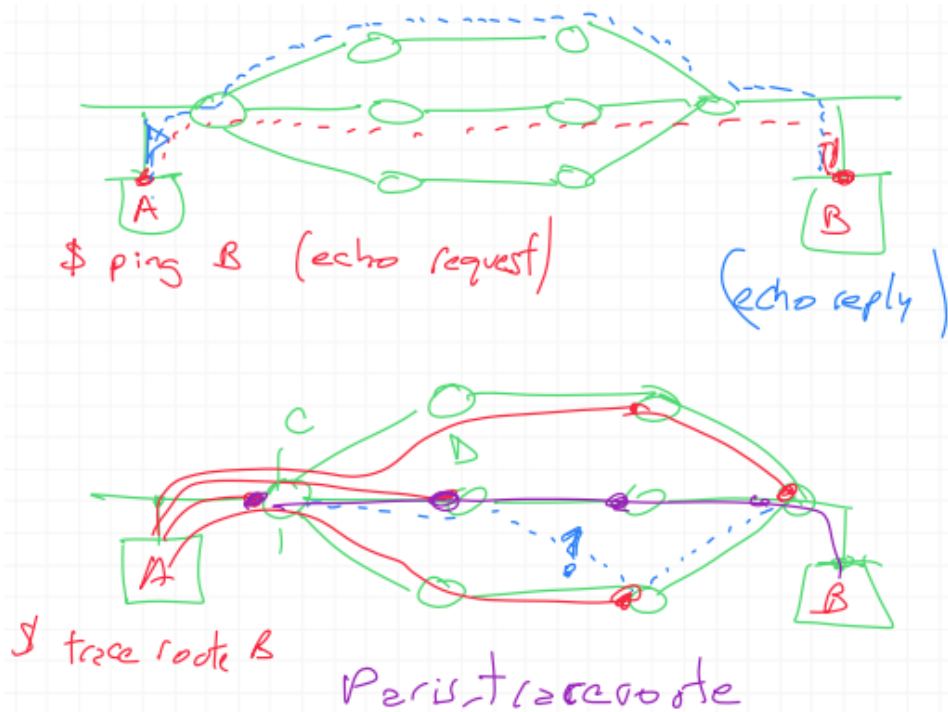
Funcionamiento PING:

Tenemos dos hosts conectados a una red, en A ejecutamos `$ping [direccionIP de B]`, este comando va a enviar paquetes ICMP con “echo request”, cuando llega al destino B conetsta con un paquete ICMP con “echo reply”. La respuesta podria ir por otro camino.

Funcionamiento Trace route:

El objetivo del trace route es. En A ejecutamos `$trace route [direccionIP de B]`, este comando va a enviar un primer paquete IP que va a llegar al primer router (TTL=1), el router puede o no responder con un paquete ICMP para informarle al destino que dropeo el paquete(el paquete tiene como direccion de origen la direccion por la cual entro el paquete). Luego volvemos a hacer lo mismo con los proximos paquetes, por lo

que no podemos suponer que los routers están enlazados, sino que lo que nos da el trace route son hops. Hay una herramienta Paristraceroute que me asegura que voy siempre por uno solo de los caminos.



Libro:

ICMP: The Internet Control Message Protocol

El protocolo ICMP lo usan los hosts y routers para comunicarse entre sí información de la capa de red. El caso de uso más típico es el de informe de errores. Los mensajes ICMP viajan dentro de los datagramas IP, como payload de IP, tienen un campo type y otro code y contienen el header y los primeros 8 bytes del datagrama IP que causó que se genere el mensaje. Traceroute está implementado con mensajes ICMP.

LIBRO:

Network Management and SNMP

Network management includes the deployment, integration, and coordination of the hardware, software, and human elements to monitor, test, poll, configure,

analyze, evaluate, and control the network and element resources to meet the real-time, operational performance, and Quality of Service requirements at a reasonable cost.

Componentes clave:

El managing server es la aplicación que controla la colección, procesamiento, análisis y/o display de la información.

El managed device una pieza del equipamiento de red que reside en la managed network. Puede ser un host, router, middlebox, modem, etc.

Toda la información recolectada se almacena en Management Information Base (MIB). Un objeto MIB puede ser un contador o el número de segmentos UDP recibidos en el host, por ej.

Un network management agent es un proceso corriendo en el managed device que se comunica con el managing server.

Un network management protocol corre entre el managing server y los managed devices.

Un ejemplo es SNMP (Simple Network Management Protocol)

[img]